

Neuromorphic Locomotion Control for a Quadruped Robot in MuJoCo using a Central Pattern Generator

Team members:

Julia Wöstmann

Lukas Stöter

Git repository URL: [LBS-Code/capstoneproject_woestmann_stoeter](https://github.com/LBS-Code/capstoneproject_woestmann_stoeter)

Individual Contributions:

- *Abstract:* Julia Wöstmann
- *Introduction:* Julia Wöstmann
- *Methodology:* Lukas Stöter
- *Results:* Lukas Stöter
- *Discussion:* Julia Wöstmann
- *Additional Background tasks:*
 - *Research into sensor stabilization:* Lukas Stöter

Abstract

In nature, animals can adapt their gait almost instantly when moving through changing environments, while robots often rely on predefined trajectories that are difficult to adjust once the environment changes. In this project, we investigate whether a biologically inspired hierarchical control architecture can improve the navigation of a simulated “ant”- robot. Our controller combines a Central Pattern Generator (CPG), which generates a rhythmic walking gait, with a higher-level finite-state controller that selects different behaviours based on sensor feedback. The controller was implemented from scratch in MuJoCo using the Gymnasium “ant”-model and tested on three different obstacles: an S-shaped maze, a free-standing wall, and uneven terrain. An unexpectedly big part of the project was to develop the perception layer that corrects and stabilizes noisy sensor readings before they are used for the actual decision-making controller. This turned out to be important, because many errors regarding the maze navigation, such as turning in open space, were caused by the robot's own movement affecting its sensors. The final controller is able to walk, navigate through the maze, move around a free-standing obstacle, and walk over uneven terrain without requiring a map or prior knowledge of the environment. Although climbing obstacles was not possible because of the physical limitations of the ant model, our project shows that combining a CPG and reliable sensor processing results in a flexible controller that can handle different environments.

Introduction

The ability to almost instantly adapt to changing environments emerges from the hierarchical interaction of rhythmic locomotion, sensory feedback and higher-level decision making (Grillner, 2006). Replicating this flexibility remains a major challenge in multi-legged robots. Traditional robot control often relies on predefined trajectories that require manual computation of controller parameter for every situation. This direct controlling of every joint with one controller is computationally expensive and often inflexible when the environment changes. While these approaches can generate stable locomotion in certain conditions, they usually struggle to generalize to unseen obstacles or unexpected terrain. In contrast, biological systems achieve robust locomotion by distributing the control over several layers instead of relying on one single controller. Rhythmic movement generation, sensory feedback and behavioural decision-making are handled separately but interact throughout the whole time, allowing animals to adapt quickly without replanning every individual movement (Ijspeert, 2008).

Biological systems make use of a hierarchical architecture for motor control by assigning different tasks to different levels of the nervous system, essentially uncoupling the processes. Higher levels, typically associated with the brain, are responsible for selecting behaviors such as walking, turning, obstacle avoidance and climbing. Lower

levels on the other hand generate the rhythmic movements required for stable locomotion. A key component of the lower-level architecture is the Central Pattern Generator (CPG), a controller capable of producing continuous rhythmic motor commands without requiring constant input from higher brain regions (Ijspeert, 2008). While Sensory feedback is not necessary for the CPG to work, it plays an important role in adapting the rhythmic movements to the environmental changes, allowing animals (and our robot) to maintain a stable walking gait over irregular terrain.

Inspired by these biological systems, CPG-based motor control has become a widely used approach in multi-legged robotics. CPGs naturally coordinate multiple limbs through coupled oscillators, producing smooth and robust gait patterns with relatively few control parameters, making them more intuitive and computationally less expensive (Ijspeert, 2008). However, CPGs alone only generate repetitive locomotion, but do not determine when the robot should change its behavior. Therefore, additional controlling mechanisms are required for more complicated tasks such as obstacle avoidance, turning, or terrain adaptation.

To address this limitation, we implemented a hierarchical hybrid control architecture. The lower level consists of a timing-driven CPG responsible for generating a stable walking gait. We selected a top-down timing-based approach, as introduced in the lecture, because it ensures reliable coordination between multiple legs while maintaining a stable walking speed (Leugering, 2026). Since our goal was robust locomotion rather than fast movement, the problem of one leg not keeping up did not arise in our setup.

Now that we had defined the lower levels of our hierarchical architecture, the robot could walk in a straight line. But as soon as it encountered an obstacle such as a box or a wall, it failed due to repeating the same movement instead of changing its behaviour. Hence, it needed an algorithm for higher-level decision making to select between various behaviors appropriate for different challenges.

Rather than replacing the CPG, we incorporated a sensor-driven finite-state controller that selects between different behaviors, including walking, turning, (probing and bracing, which currently do not work) based on sensory feedback from the environment. This creates a hybrid control system, combining continuous locomotion dynamics generated by the CPG with discrete behavioral modes. By separating the decision-making process from motion generation, the controller became easier to extend based on changing environments.

Within the limited scope of this project, we apply this hierarchical architecture to a simulated “ant” robot capable of navigating structured environments. The robot shows stable forward locomotion, obstacle avoidance in a small maze, navigation around free-standing walls, and walking over uneven terrain.

Methodology

The simulation is built in MuJoCo on the standard ant model from the Gymnasium library: a central torso with four legs, each with two hinge joints driven by position actuators — a hip that swings the leg forwards and backwards, and an ankle that raises and lowers it [Gymnasium]. The starting scaffold adds the environment the ant must handle — an S-shaped maze, a free-standing wall and a patch of uneven terrain — together with the sensors it uses to perceive its surroundings and its own pose.

Control architecture

The code in our notebook is written from scratch, using AI and Mujoco's own website [Mujoco] to help research mathematically more complex approaches. Structurally it follows Lecture 8 and Albus's formulation of hierarchical control, in which a goal is decomposed into subtasks passed down to the next lower level, with higher layers slow and deliberative and lower layers fast and reactive.

That division is the shape of the program. `run_simulation` is the slow layer: each tick it senses the surroundings, hands control to whichever behaviour owns the current mode, and steps the physics. It knows nothing about mazes, walls or boxes. Decision-making is split in two — perception in `sense_environment()`, action selection in `behaviour()`, which is only a dispatcher mapping the current mode to the function that owns it. The decision logic lives in those behaviour functions, so a new obstacle type means a new function rather than a change to the runner. Every mode change is printed, which was a development aid but also makes the structure easy to follow.

Perception and sensor conditioning

`sense_environment` is the single place sensors are read. It receives the model, its data and `movement_state` (the values relevant to each type of navigation, created by `make_movement_state` and updated as the run proceeds) and returns the current cardinal heading, a stabilized front distance, trustworthy left and right distances, a boolean `open_sky` and a slope level.

Conditioning is necessary because the rangefinders are bolted to a body that yaws, rolls and pitches with every step, so the raw values are often misleading. A few things need mentioning:

1. A miss returns -1 , which reads as an obstacle right on top of the ant. `read_range()` turns it into $+\infty$, so "nothing there" means infinitely far. This matters most for `open_sky`, which requires both side rays to be clear — exactly the case in which both of them miss.
2. An angled ray measures a slant range, not the perpendicular distance the controller reasons about. `stable_side_distance()` and `stable_front_distance()` project it back using yaw and roll (or pitch, at the front), read from the torso quaternion by `get_yaw()` and

get_roll_pitch()). The correction goes through cos_clamped(), which limits the angle to $\pm 30^\circ$ so a badly tilted ant cannot scale every reading towards zero and see obstacles all around itself.

3. A badly angled ray measures the wrong surface entirely. What this means is that the movement of the ant regularly led to a sensor missing the intended obstacle and instead detecting a different one entirely. But because the reading only appears slightly irregular and not implausible, we can't solve this geometrically. So we refuse: for values beyond a threshold for the wall distance "ALIGN_TOLERANCE", stable_side_distance() returns None and the caller keeps its last trusted value (right_belief / left_belief).

4. A single reading, however well corrected, is a momentary condition. sense_environment therefore turns readings into conclusions by requiring persistence. An exponential moving average builds a continuous belief only once a condition genuinely holds (open_sky_level, slope_level); two thresholds instead of one keep a boolean from flickering at the boundary (open_sky); and condition_held() requires a trigger to stay true for a minimum time before the state machine may act on it.

Action selection — behaviours

Where perception answers what is around the ant, the behaviours answer what to do about it. All of them are built from the same primitives, which is what keeps them comparable.

Steering. hold_heading holds a cardinal direction; wall_following_steer biases it by the distance to the wall being followed; lane_steer biases it by the offset from a target lane. The latter two are cascaded rather than summed — the distance error becomes a small heading offset that the heading controller then chases — because two steering terms acting on one body make the ant weave. heading_steer is proportional but lifted over a measured dead band, since any command below roughly 0.25 produces no yaw at all. Without heading hold the ant cannot walk straight: the gait yaws about $-0.6^\circ/\text{s}$ by itself, a 30° drift over one corridor.

Turning. Pivots turn until the measured yaw reaches an absolute target heading, not until 90° have been swept. This was the single most persistent failure of the project. A relative turn never ends exactly on 90° , and in a relative scheme each turn takes the previous orientation as its zero, so the small residuals add up — after four corners the ant can be $15\text{--}20^\circ$ off the direction it believes it is walking in, without ever noticing. Since the desired heading is what the sensor conditioning projects against, a heading that is quietly wrong corrupts the readings meant to correct it. The fix is an absolute frame: the initial yaw is snapped to the nearest cardinal, and every heading afterwards is an absolute multiple of $\pi/2$ (snap_to_cardinal), so the error at any corner is bounded by the pivot tolerance instead of accumulating.

Recovery - `unstuck_check` watches whether the torso is actually moving; if the ant walks without progressing — wedged on something no ray can see — the gait runs in reverse for a moment and then resumes the interrupted mode. The thresholds are relaxed on terrain, where genuine progress is slow enough that the ordinary test fired when the ant was making slow progress.

On top of these sit four behaviours:

Walk - is the default that every other behaviour hands back to: it holds the home heading, out in the open also a lane, and watches for a decision becoming due. Once a front obstacle is detected and debounced, `classify_obstacle` uses height to decide whether it must be walked around, and `open_sky` whether to treat it as a corridor or a free-standing obstacle — keeping one hand on the wall is right in the first case and would orbit forever in the second. Sustained rising ground is read as terrain to cross, not an obstacle to avoid.

maze_* - follows the right-hand rule in three sub-modes: follow the wall on the right, pivot left when the front is blocked, and, when the wall ends, walk on a measured distance before turning right so the legs clear the corner. Once `open_sky` holds there is no corridor left and control returns to walk.

wall_* - rounds a free-standing obstacle with the same stages mirrored, since it may be passed on either hand; `choose_go_around_side()` picks that hand during the approach. Unlike the maze follower it knows when it is finished — after the second corner `lateral_offset()` measures the return to the line the ant left.

terrain_* - almost empty: a slower gait cycle, selected by `run_simulation` from the mode, plus holding the lane so the ant tracks mostly straight instead of drifting downhill, although it still struggles with the hills somewhat.

Locomotion layer

Underneath every behaviour sits `cpg_controller`, the only place actuator values are produced. Four coupled phase oscillators, one per leg, advance at a common frequency and are pulled back towards their intended phase relationships by a sine coupling term, so a leg disturbed by rough ground rejoins the pattern instead of drifting out of it, which is why the gait is generated rather than scripted. Each phase maps to a hip and ankle target through a three-part cycle (swing, a short touchdown hold, stance sweep), mirrored per leg by sign factors, and a PD controller tracks those targets joint by joint. Steering is differential at this level: a steer value in $[-1, 1]$ reverses the legs on one side in proportion to its magnitude, so the same gait yields anything from a straight walk to an in-place pivot, and a direction argument runs the cycle backwards. Because every behaviour expresses itself as that single scalar, the layers above never deal with legs or joint angles.

Results

Development ran as a sequence of experiments, each prompted by the failure of a previous one.

Locomotion came first: walking in a straight line. It fixed the gait constants and produced the measurement that shaped everything above it, the gait yaws about $-0.6^\circ/\text{s}$ on its own, so straight walking is not a property of the gait but something a controller must maintain.

Box climbing came second and failed, for anatomical rather than algorithmic reasons: with hips limited to $\pm 30^\circ$ the ant cannot place a foot on the obstacle and then shift its weight over it. This first negative result redirected the project from climbing to navigating around obstacles.

Navigation was the largest series, run as isolated maneuvers, one approach, one pivot, one wall-following pass, to fix the constants the body's geometry dictates. This is why each navigation constant is annotated with the measurement behind it rather than a plausible value:

- The feet reach 0.88 m sideways and a quarter turn drifts the body ~ 0.45 m forwards, which sets the wall-following and front-trigger distances. The earlier 0.5 m front threshold fired only once the legs were already against the wall
- Corner clearance: 1.2 m made the ant jam during turns; a slight increase to 1.6 yielded no significant improvement; 2 m clearance worked well
- Telling a wide corridor from open space: A range of 10m as threshold for classification worked well regarding the mazes size, lower values made the ant falsely claim it had escaped the maze
- Choosing a side to pass on: without the alignment gate the angled sensors were inaccurate, because yaw swings the two rays in opposite directions and the error is systematic

Terrain was only recently added and therefore remains quite rudimentary. Lowering the normal gait frequency from 0.87 to 0.55 seemed to help and this is as far as we have come. A large part of why it remotely works right now is the CPG phases correcting themselves if one leg is somewhat blocked.

Reproducibility

The run is deterministic: the initial pose is fixed in the model file, the oscillator phases start from fixed offsets, and no noise is injected anywhere. The final cell is the whole experiment — `run_simulation(duration=400.0, capture_fps=3, playback_fps=30)` — which prints one line per mode change and returns the summary metrics above. That log is the primary record: since every deliberative decision is a mode change, it is a complete account of what the controller decided and on what evidence.

Any experiment was tested by modifying the ants starting position in the xml, placing it in front of manually selected (relevant) obstacles.

Relation to the problem

The run supports the claim the architecture was built on: a single controller, without learning, a map or any global knowledge of the course, gets through three structurally different obstacles by decomposing navigation into subtasks and switching between them on sensory evidence. While the ant movement() is generally very slow, our program works well for our specific course and is robust to some changes. The experiments also show where the difficulty actually lies: almost every failure traced back to a rangefinder bolted to a body that the gait itself swings, returning not noise but a plausible reading about the wrong surface. Correcting readings geometrically, refusing to report when the geometry seems disturbed, and requiring evidence to persist, makes a purely reactive lower layer viable on this body. Box climbing failed for anatomical reasons no controller can overcome and we chose not to change its anatomy, preserving existing code logic made for a 4-legged ant and continuing with the initial challenge posed by the model.

Discussion

Overall, the project achieved its original goal of implementing a biologically inspired controller that allows the ant to navigate several different environments without requiring a predefined trajectory or a map. The combination of a low-level CPG with a higher-level finite-state controller proved to be a suitable architecture for this task. The CPG generated the wanted stable walking gait, while the behaviour layer allowed the robot to react to different situations by switching between behaviors based on the input it received from its sensors. Separating locomotion from decision-making also made the controller easier to extend, since new behaviours could be added without modifying the gait generation itself.

One of the biggest challenges during the project was the perception layer. Initially, we expected most of our work to focus on implementing different navigation strategies. Instead, many failures were due to noisy or misleading sensor readings caused by the movement of the robot itself. Small orientation errors were enough to make the range sensors report plausible but incorrect distances due to the slightly wobbly walk, which then caused wrong behavioural decisions. As a result, a significant part of the project shifted towards stabilizing and validating sensor data before it reached the decision layer.

Another big part was the failure of the box climbing experiments. We initially expected that climbing could be achieved by modifying the gait or changing the timing of the CPG. However, after many attempts and fails, testing showed that the limitations came from the physical design of the ant rather than from the controller. We deliberately chose to

keep the original anatomy and focus on robust navigation around obstacles, which gave us way better results.

Although the controller performs well in the tested environments, it still has several limitations. The behaviours are entirely hand-designed, so an idea for future models would include a neuronal network that autonomously learns new behaviors, without having to define them individually. The maze-solving behaviour also assumes an S-shaped maze and follows a simple right-hand rule, which would not solve all possible maze layouts. Furthermore, the controller has no memory or global representation of its surroundings and therefore cannot plan efficient routes or learn from previous experience.

Conclusively, this project changed our understanding of robot control. At the beginning, the focus was mainly on generating a walking gait and implementing obstacle avoidance. But during development, we encountered many unexpected challenges along the way, which required us to dive deeper into motor control and especially sensor stabilization.

References

S. Grillner (2006), *Biological Pattern Generation: The Cellular and Computational Logic of Networks in Motion*. Volume 52(5), 751-766. Accessed: 20.07.2026.

A.J. Ijspeert (2008), *Central pattern generators for locomotion control in animals and robots: A review*. Neural Networks, Volume 21(4), 642-653. Accessed: 20.07.2026.

J.M. Leugering (2026), *Lecture 8: Hierarchical and hybrid control. Introduction to Neuromorphic Control (SoSe 2026)*

MuJoCo Documentation. *MuJoCo Overview*. Available at:
<https://mujoco.readthedocs.io/en/stable/overview.html> (accessed over the whole project).

Gymnasium Documentation. *Ant Environment*. Available at:
<https://gymnasium.farama.org/environments/mujoco/ant/> (accessed over the whole project).